

THAI NGUYEN UNIVERSITY
OF INFORMATION AND COMMUNICATION TECHNOLOGY
INSTITUTE OF INTERNATIONAL TRAINING



Software Deployment and Maintenance Course

Report

Project Title: Software Packaging and Release Process

This submitted to

Lecturer: Quach Xuan Truong

By

BUI THU HUONG

LUU THE HA

Student ID: DTC235280062

Student ID: DTC235340007

November 11, 2025

TABLE OF CONTENTS

I) OVERVIEW.....	3
1.1. Topic Importance.....	3
1.2. Reasons for Choosing the Topic.....	3
1.3. The Scope.....	4
1.3.1. Packaging.....	4
1.3.2. Deployment.....	4
II) PACKAGING AND RELEASE METHODS.....	5
2.1 Web Application Packaging Methods.....	5
2.1.1. Front-end Packaging (SPA/Static Sites).....	5
2.1.2. Back-end and Microservices Packaging (Dynamic Apps).....	5
2.2. Web Application Release Methods.....	6
2.2.1. Front-end/Static File Deployment.....	6
2.2.2. Back-end/Service Deployment (Dynamic Service Deployment).....	6
III) TOOLS FOR PACKAGING AND RELEASING (CI/CD PIPELINE).....	8
CONCLUSION.....	9
References.....	10

I) OVERVIEW

1.1. Topic Importance

Software packaging and deployment is the critical final stage of the software development life cycle (SDLC) that makes an application available and usable for its end-users. Its importance lies in bridging the gap between a finished piece of code and a functioning, reliable product in a real-world environment.

- **Reliability & Consistency:** It ensures that the software works the same way everywhere, regardless of the user's machine or server environment. This prevents the common "it works on my machine" problem.
- **Efficiency:** A streamlined process allows for rapid, repeatable, and automated delivery of new features, bug fixes, and security patches.
- **User Experience:** A smooth installation and update process is often the user's first interaction with the software. A bad experience can lead to frustration and product abandonment.
- **Risk Management:** Formal deployment strategies (like blue-green or canary deployments) allow teams to release updates with minimal downtime and provide a way to quickly roll back if problems are found.

1.2. Reasons for Choosing the Topic

This topic is a common focus for both businesses and individuals (like students or developers) because it represents a critical, practical, and highly in-demand skill set.

- **For Businesses:**
 - **Speed to Market:** Efficient deployment (often via CI/CD pipelines) is a major competitive advantage, allowing companies to innovate faster.
 - **Scalability:** Modern packaging (like containers) and deployment (like to the cloud) are essential for building applications that can serve millions of users.
 - **Cost & Resource_Management:** Automation in this area reduces the chance of human error and frees up developer time from manual, repetitive tasks.
- **For Individuals (Developers/Students):**
 - **Practical Skill:** Knowing how to build software is only half the battle. Knowing how to *ship* it is what makes a developer effective in a professional setting.
 - **DevOps Culture:** This topic is the core of DevOps, which unites development (Dev) and operations (Ops). Understanding it is essential for working in modern tech teams.

- Managing Complexity: As projects grow, managing dependencies (the other libraries your code needs) becomes complex. Packaging is the solution to this complexity.

1.3. The Scope

The scope of "Package and Deploy" covers all activities required to take completed code and deliver it to a target environment. It can be broken into two main phases:

1.3.1. Packaging

This is the process of bundling the application's code along with everything it needs to run into a single, standardized unit.

The scope includes:

- Dependency Management: Identifying and including all necessary libraries, frameworks, and other dependencies.
- Configuration: Bundling default configuration files (e.g., settings for database connections, API keys).
- Compilation: Compiling source code into executable files (if necessary for the language).
- Artifact Creation: Creating the final package file. This could be a:
 - Container image (e.g., a Docker image)
 - Installer (e.g., .msi for Windows, .pkg for macOS)
 - Compressed archive (e.g., .zip, .tar.gz)
 - Application bundle (e.g., .apk for Android, .ipa for iOS)

1.3.2. Deployment

This is the process of distributing and activating the package in the target environment (such as a user's computer, an on-premise server, or the cloud).

The scope includes:

- Release Planning: Deciding *when* and *how* to release (e.g., all at once, to a small group of users first).
- Distribution: Moving the package to the target environment.
- Installation & Configuration: Running the installer or container and applying the correct environment-specific settings (e.g., setting the production database password).
- Testing & Verification: Performing final checks in the production environment to ensure everything is working as expected.

- **Monitoring & Maintenance:** The process doesn't end at launch. This also includes monitoring the application for errors and performance, as well as managing future updates and patches.

II) PACKAGING AND RELEASE METHODS

2.1 Web Application Packaging Methods

The goal of packaging is to create a deployable Artifact. The packaging method primarily depends on whether the application is Front-end (static) or Back-end (dynamic).

2.1.1. Front-end Packaging (SPA/Static Sites)

For Single Page Applications (SPAs) built with React, Vue, Angular, or other Static Sites:

- **Process:**
 1. **Build/Bundle:** Use tools like Webpack, Vite, Rollup, or Parcel to compile source code (JSX/TSX/Vue files) into pure JavaScript, CSS, and HTML code.
 2. **Optimization:** Perform optimization steps such as Minification (reducing file size by removing unnecessary characters), Tree Shaking (eliminating unused code), and Hashing (adding a hash to file names to support caching).
 3. **Artifact:** The final product is a collection of static files (e.g., index.html, main.js, style.css) stored in a single directory (usually dist or build).
- **Packaging Result:** A directory containing static files ready to be served by a web server.

2.1.2. Back-end and Microservices Packaging (Dynamic Apps)

For Back-end services, APIs, or Multi-Page Applications (MPAs):

- **Traditional Method (Separate Artifacts):**
 - Use the language-specific build manager (e.g., Maven/Gradle for Java to create .jar or .war files, or pip/Poetry for Python) to generate a bundle containing the executable code, configuration files, and all dependencies.
 - **Drawback:** Requires ensuring that the server environment (operating system, language/runtime version) precisely matches the build environment.

- Modern Method (Containerization - Docker):
 - Dockerfile: Use a Dockerfile to precisely define the necessary environment (Base OS, runtime version, required binaries).
 - Build Image: The Docker tool reads the Dockerfile to create a Docker Image. This Image includes the packaged application AND its entire runtime environment.
 - Advantage: Ensures complete consistency ("it runs on my machine, it will run in Production").
- Packaging Result: Docker Image (stored in a Docker Registry such as Docker Hub, ECR, GCR).

2.2. Web Application Release Methods

Releasing is the phase of automating the deployment of the packaged Artifacts into the actual runtime environment (Production). Modern release strategies help minimize risk and downtime.

2.2.1. Front-end/Static File Deployment

- Mechanism: Typically involves uploading the packaged static files to a storage service.
- Methods:
 - CDN (Content Delivery Network) Deployment: Push Front-end files to services like Cloudflare, Amazon S3 + CloudFront, or Netlify/Vercel. CDNs not only store but also distribute content to users from the nearest location, resulting in high-speed page loads.
 - Atomic Deployments: Ensures that all new files are successfully uploaded to a new directory before the web server's path (symlink) is updated. If an error occurs during upload, it has no impact on the end-user.

2.2.2. Back-end/Service Deployment (Dynamic Service Deployment)

This is where advanced deployment strategies are applied to manage risk and achieve zero-downtime.

Strategy	Brief Description	Benefits	Risks
Recreate	Stops all old instances, then deploys the new version.	Simple, easy to implement.	Causes downtime for users.
Rolling Update	Replaces old application instances with the new version gradually (in batches).	No downtime, easy to monitor for errors.	Requires careful configuration management (Backward Compatibility).
Blue/Green	Maintains two identical Production environments. The old version runs on "Blue." The new version is deployed to "Green" (inactive). Once testing is complete, traffic is instantly switched from Blue to Green.	Instant deployment, Instant Rollback (simply switch traffic back to Blue).	Resource-intensive (must maintain two environments).
Canary Release	Deploys the new version to a small percentage of users (e.g., 1-5%). Monitors performance metrics and errors. If stable, it gradually increases the percentage to 100%.	Maximizes risk reduction by testing on a small, real user segment.	Complexity in managing and routing traffic.

III) TOOLS FOR PACKAGING AND RELEASING (CI/CD PIPELINE)

The entire Packaging and Release process is managed by a CI/CD Pipeline.

Phase	Common Tools	Role
Source Code	GitHub	Stores source code and triggers the Pipeline upon changes (Commit/Merge).
Continuous Integration (CI)	GitHub Actions	Automates the process of Build, Test, and Packaging (Docker Image creation).
Artifact Storage	Docker Registry (Docker Hub, AWS ECR)	Stores packaged products (Docker Images, .jar files, Front-end static files).
Continuous Delivery (CD)	Terraform	Automates the Deployment of the artifact from storage to the server environment.

CONCLUSION

In conclusion, in a competitive digital landscape, a robust and fully automated packaging and release pipeline is essential for sustaining development speed, ensuring product quality, and delivering a seamless user experience. Automation transforms packaging and releasing from a costly, high-risk chore into a strategic business advantage. It empowers the business to iterate faster than its competition while simultaneously delivering a more reliable and higher-quality product.

References

1. [Overview of Software Release Process in Software Engineering - GeeksforGeeks](#)
2. [What is CI/CD? - GeeksforGeeks](#)
3. [haluu07/final](#)